

Advanced & Emerging RL Methods

Reinforcement Learning

Advanced & Emerging RL Methods

- Learning with models and fixed data: model-based RL, offline RL
- Learning with demonstrations and humans: behavior cloning, imitation learning, RL from human feedback
- RL as sequence and generative modeling: trajectory modeling, decision transformer, diffusion policies
- Scaling RL to real-world robotics: embodied RL, sim-to-real gap, generalist robot policies, vision-language-action models

Idea of the Chapter

high-level overview of RL approaches that extend or complement standard model-free methods

- model-based techniques
- offline and data-driven approaches
- imitation learning
- generative or multimodal policy methods

Model-Based and Offline RL

Model-Based RL

instead of only a policy or a value function, learn a dynamics model $p(s_{t+1}, r_t | s_t, a_t)$ predicting future states and rewards

the model can then be used for

- planning: simulate possible futures and select actions (e.g., MPC)
- data generation: create imagined rollouts for policy learning
- improved sample efficiency: reduce expensive interaction with real environment

key challenge: model errors can accumulate over long horizons

→ inaccurate predictions may degrade policy performance

Offline RL

learn a policy entirely from a fixed dataset:

- training data consists of previously collected trajectories
- no online exploration or environment interaction during learning

key idea: use reward information to learn policies that may prefer higher-return behaviors in the dataset

important caveat:

improvement only possible within the coverage of the dataset

→ limited generalization to unseen/out-of-distribution actions

Learning from Demonstrations

Behavior Cloning

learn a policy by directly imitating demonstrations:

- training data consists of expert trajectories
- learn a mapping from states to actions

→ treat RL as a standard supervised learning problem (no reward, no exploration)

effective when expert data is high quality and covers relevant states

simplest form of imitation learning

Imitation Learning

behavior cloning: direct supervised learning on state-action pairs

imitation learning: broader framework of learning from demonstrations (including methods that go beyond simple supervised learning)

useful when exploration is difficult or unsafe

key limitation: performance typically bounded by the quality of the expert

→ cannot reliably exceed expert without incorporating reward-based learning (e.g., offline RL)

RL from Human Feedback

example for supporting
large language models
(transformers) with RL

used in famous ChatGPT

goal: improve alignment
with user intentions

→ learn from human
preferences

Step 1

**Collect demonstration data,
and train a supervised policy.**

A prompt is
sampled from our
prompt dataset.

Explain the moon
landing to a 6 year old

A labeler
demonstrates the
desired output
behavior.

Some people went
to the moon...

This data is used
to fine-tune GPT-3
with supervised
learning.

SFT

Step 2

**Collect comparison data,
and train a reward model.**

A prompt and
several model
outputs are
sampled.

Explain the moon
landing to a 6 year old

A B
Explain gravity... Explain war...
C D
Moon is natural satellite of... People went to the moon...

A labeler ranks
the outputs from
best to worst.

D > C > A = B

This data is used
to train our
reward model.

RM

Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.

Write a story
about frogs

The policy
generates
an output.

PPO

Once upon a time...

The reward model
calculates a
reward for
the output.

RM

The reward is
used to update
the policy
using PPO.

r_k

RL looks at reward of text output passages as a whole
(rather than token-level loss in supervised learning)

[source](#)

RL as Sequence and Generative Modeling

Trajectory Modeling

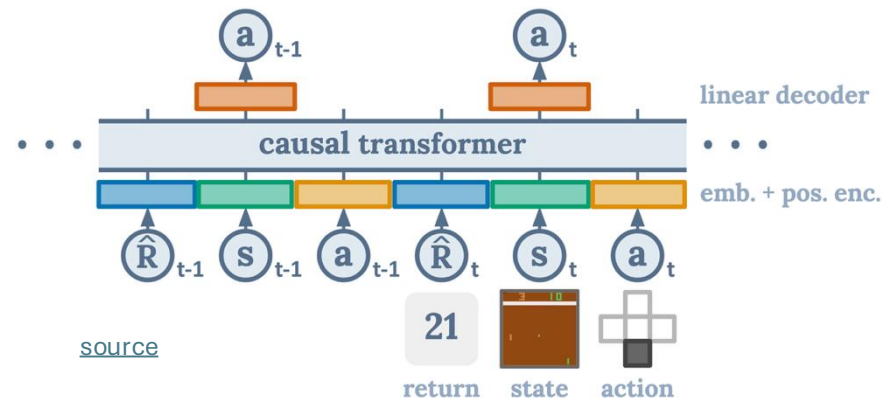
reframe RL as sequence generation:

- a trajectory is treated as a sequence: $(s_1, a_1, r_1, s_2, a_2, r_2, \dots)$
- predict next action given past states, actions, and returns
- instead of learning “policy + value function”, learn a sequence model over behavior
- uses architectures from sequence modeling (e.g., transformers)
- naturally handles long-horizon dependencies
- enables offline training on large trajectory datasets
- foundation for generalist policies in robotics and control

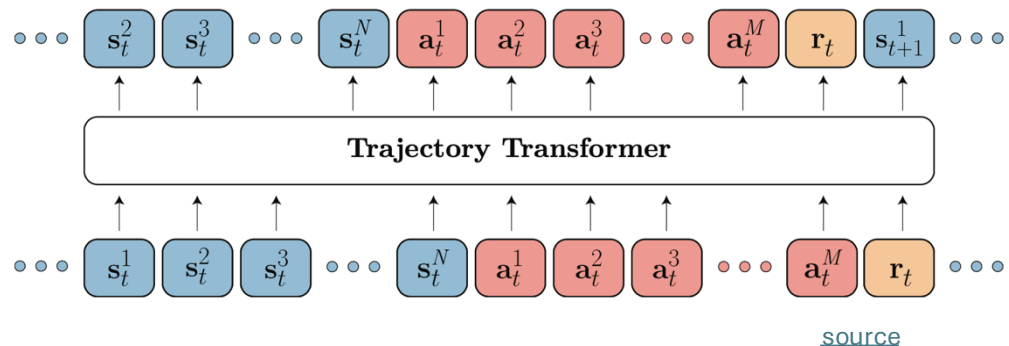
Decision Transformer

generative: transformer decoder architecture to autoregressively model trajectories
credit assignment directly via self-attention: implicitly forming state-return associations via similarity of query and key vectors (maximizing the dot product)
desired return tokens as prompt for action generation → favor actions that appear in high-return trajectories

Decision Transformer: conditioning on desired return, past states and actions to generate future actions



Trajectory Transformer: predicting also states and returns (adding model-based components, planning with beam search)



Diffusion Policies

trajectory modeling: generates actions autoregressively over time

diffusion policies: generate each action individually from a conditional distribution $p(a_t|s_t)$ via an iterative denoising process

- handles complex, high-dimensional multi-modal action spaces
- naturally supports vision-based control policies
- strong performance in imitation learning settings

Scaling RL to Real-World Robotics

Embodied RL

agents (robot, avatar, simulated agent) interacting with complex, realistic environments (e.g., homes, warehouses, outdoor scenes)

→ must deal with:

- partial observability
- physical constraints
- long-horizon interactions

Isaac Sim & Lab

high-fidelity robotics simulation and learning platform for training and evaluating embodied AI agents at scale

- Isaac Sim (simulation engine): realistic modeling of rigid body dynamics, contacts and collisions, sensors (RGB, depth, LiDAR)
- Isaac Lab (RL/learning framework): built on top of Isaac Sim for robot learning workflows

enables massive data generation in simulation, reducing reliance on expensive real-world robot data

Sim-to-Real Gap

policies trained in simulation often fail to transfer directly to the real world due to mismatches between simulated and physical environments

→ limits direct deployment of RL policies trained in simulation (central challenge in robotics and embodied AI)

common strategies to bridge the gap:

- domain randomization: vary simulation parameters during training
- system identification: calibrate simulation to better match real dynamics
- fine-tuning on real data: small amount of real-world adaptation
- robust policy learning: train policies that generalize across variations

Generalist Robot Policies

idea: learn a single policy that can perform many different robotic tasks, rather than training separate models per task

→ one model, many skills: manipulation (pick & place, grasping) navigation, tool use, multi-step household tasks

- large-scale multi-task datasets (often across robots and environments)
- unified policy architectures (often transformer-based)
- conditioning on task instructions (e.g., language, goal images, demonstrations)

step toward scalable robot foundation models

Vision-Language-Action (VLA) Models

idea: learn robot policies that map visual and language inputs directly to actions, enabling instruction-following embodied behavior

→ unify perception, reasoning, and control in a single multimodal policy model

model components:

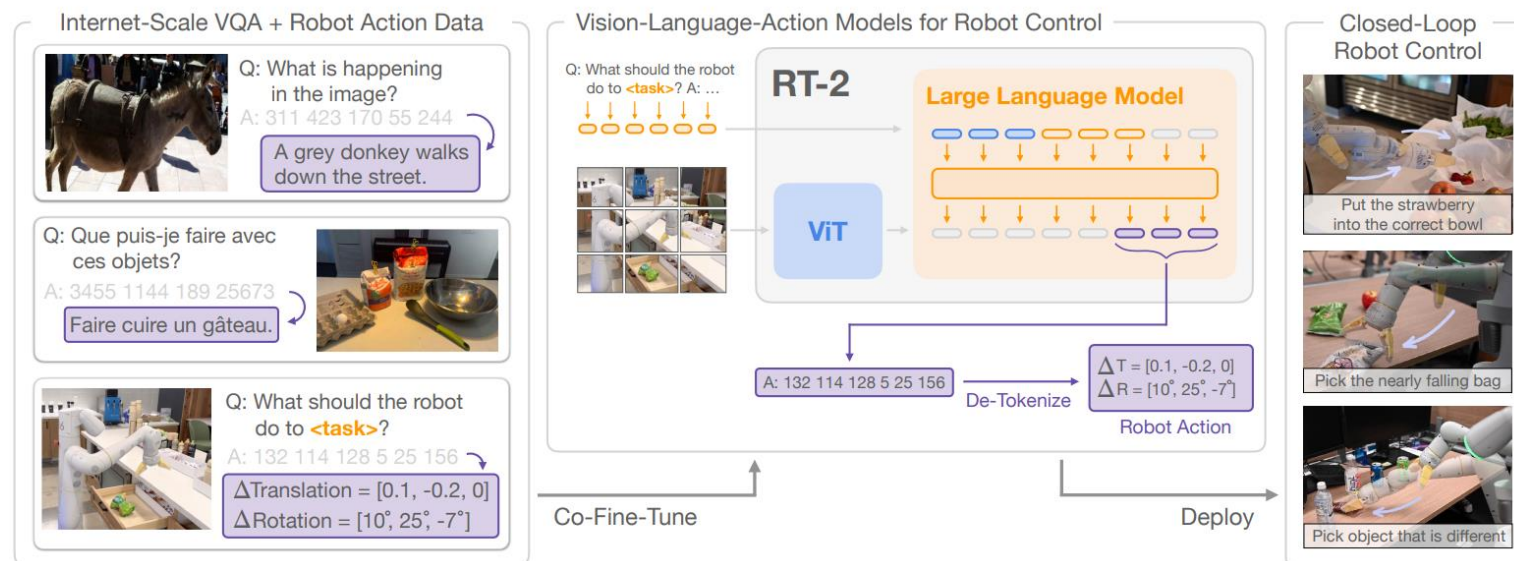
- vision encoders (e.g., ResNet, ViT): extract features from images / observations
- language models: encode task instructions or goals (natural language control)
- multimodal fusion: combine vision + language into a shared representation
- action head / decoder: outputs motor commands or action tokens

central component in modern robot foundation models

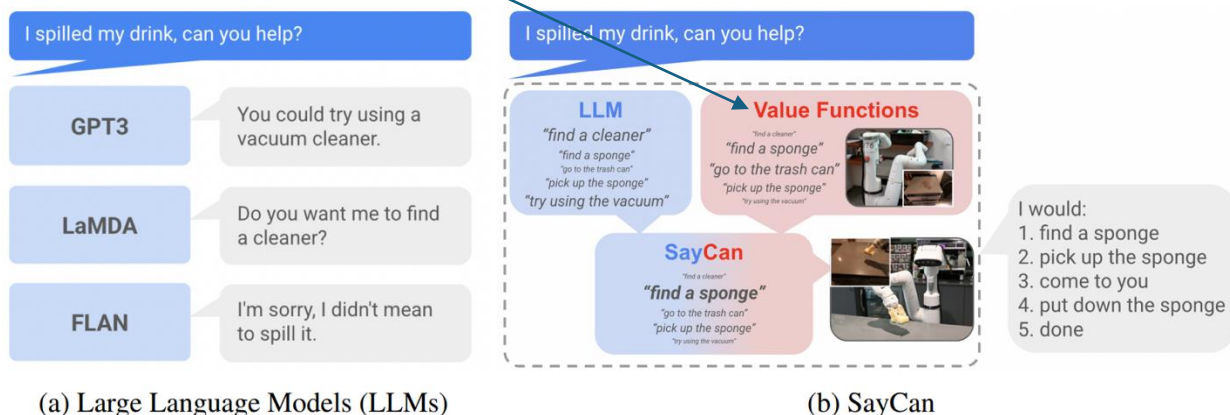
Robotic Control via LLMs (and Vision)

RT-2: vision-language-action model learning from web and robotics data

- representation of actions as tokens
- generalization by using pre-trained vision-language models



grounding with pre-trained skills (**SayCan**):



Code as Policies:

